Project - Low Level Design

On

Dockerized Construction Services

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Ansh Kumbhare	EN22CS301140
2.	Anshul Sen	EN22CS301158
3.	Anshul Yadav	EN22CS301159
4.	Armi Bhawsar	EN22CS301194
5.	Athrav Dubey	EN22CS301229

Group Name: Group06-D2

Project Number: DO-18

Industry Mentor Name:

University Mentor Name: Prof. Akshay Saxena

Academic Year: 2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. References
4. GitHub

1. Introduction

This document presents the system design for the Dockerized Construction Service, developed as part of a DevOps-based project. The project demonstrates a complete containerization workflow using Docker along with CI/CD automation.

The application represents a Construction Planning Service integrated with GenAI capabilities, where intelligent modules assist in planning, scheduling, and resource estimation using AI-driven insights.

The DevOps pipeline ensures the application is:

- Consistently packaged
- Easily deployable
- Scalable across environments
- Secure and optimized

1.1. Scope of the document

This document includes:

- Docker-based containerization strategy
- Multi-stage Docker build optimization
- Docker Compose orchestration
- CI/CD pipeline using GitHub Actions / Jenkins
- Environment and dependency management
- Security and performance considerations
- API catalogue and service endpoints

This document does not cover deep internal AI model logic, only deployment and system design.

1.2. Intended Audience

This document is intended for:

- DevOps Engineers
- Backend Developers
- AI Engineers

- Evaluators
- Future Maintainers

1.3. System overview

The system consists of two main repositories:

Repository 1 – Construction Service(Application)

Contains:

- Backend logic (construction planning)
- GenAI Modules (task suggestions, cost estimation)
- APIs and business logic

Repository 2 – DevOps Infrastructure

Contains:

- Docker file
- Docker compose
- CI/CD pipeline
- Environment configs

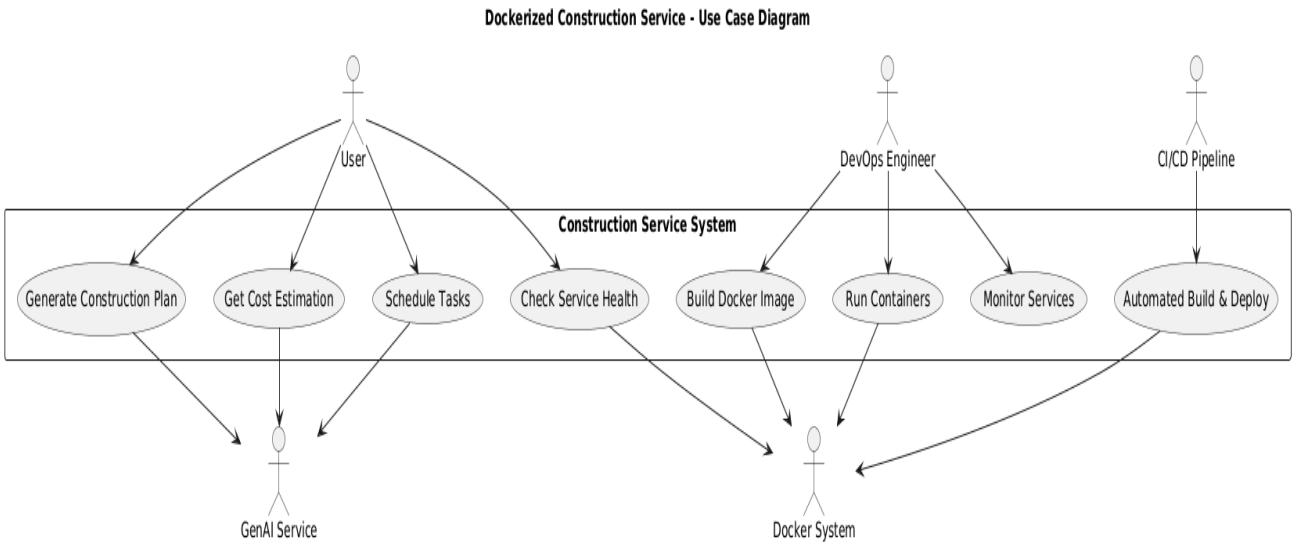

Figure 1: Use Case Diagram

High-Level Workflow

- Developer pushes code → GitHub
- CI/CD pipeline triggers
- Docker image is built using multi-stage build

- Image is stored in Docker Hub
- Docker Compose deploys services
- Application becomes accessible

2. System Design

Architecture Flow

User → API Service → Container → Database → Response

↓

CI/CD Pipeline

↓

Docker Registry

↓

Kubernetes Cluster

2.1. Application Design

The application consists of the following modules:

1 Frontend Layer

- UI for users (Web Interface / API Client)
- Accepts construction-related queries

2. Backend Layer

- Business logic (Construction workflows)
- Handles scheduling, resources, cost estimation

3. GenAI Layer

- AI-powered planning assistant
- Suggests:
    - Task sequences

- Resource allocation
- Risk analysis

2.2. Process Flow

The system follows a structured execution flow:

- User sends request (e.g., "Create construction plan")
- Backend processes input
- GenAI module generates intelligent suggestions
- Backend validates and structures output
- Response returned to user

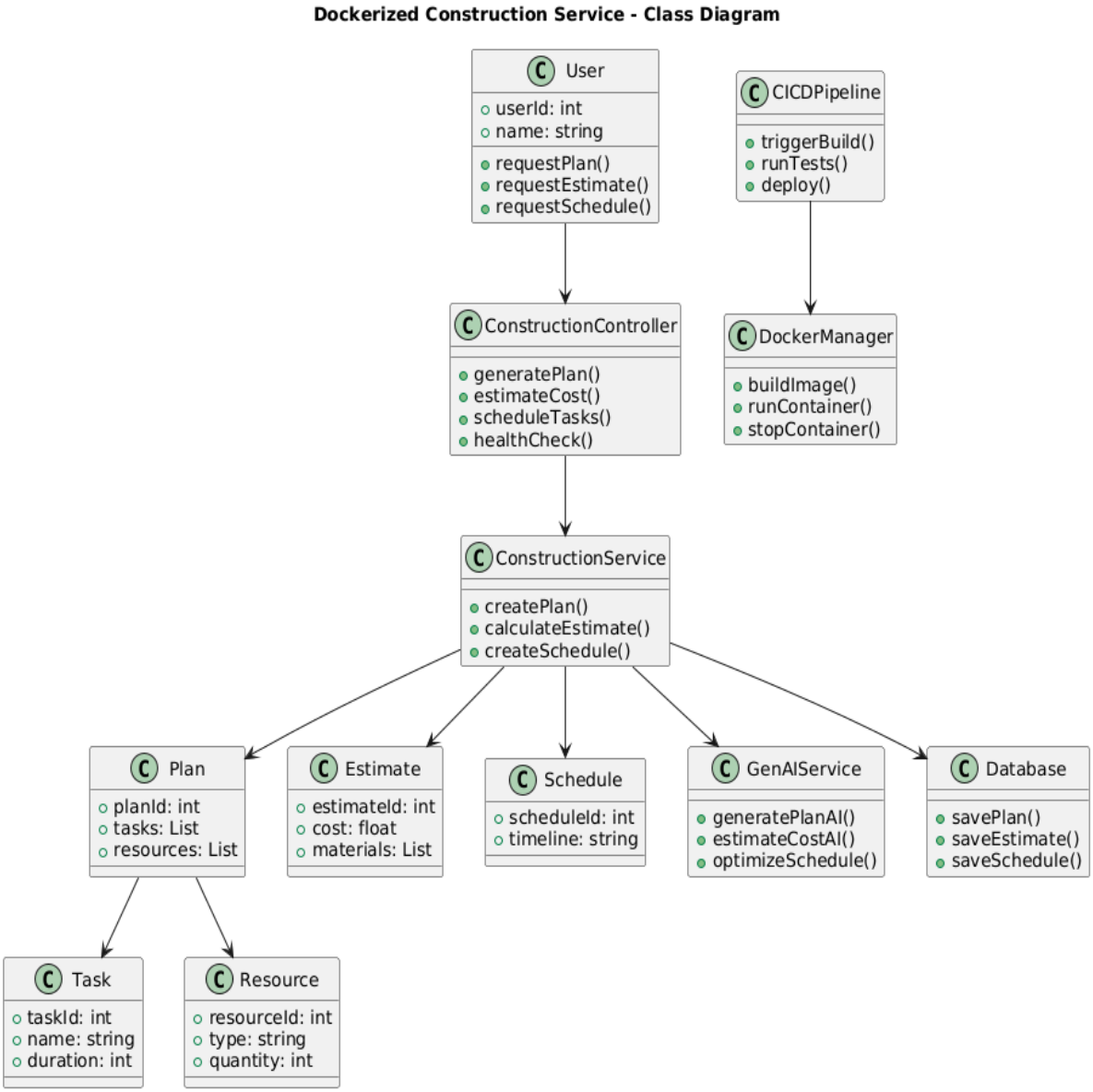

Figure 2: Activity Diagram

2.3. Information Flow

The flow of information in the system is as follows:

User → Frontend → Backend → GenAI Module → Backend → Frontend → User

2.4. Components Design

- **Component 1 – Docker file**

Property	**Value**
Base Image	python:3.11-slim
Build Type	Multi-stage build
Working Directory	/app
Exposed Port	8000
Entry Point	python app.py

- **Component 2 – Docker Compose**

Service	Description
app	Main construction service
db	Database(PostgreSQL/MySQL)
cache	Redis (optional)

- **Component3 3 – CI/CD Pipeline**

(using GitHub Actions / Jenkins)

Step	Action
1	Checkout code
2	Install dependencies
3	Build Docker image
4	Run tests
5	Push image to Docker Hub
6	Deploy using Docker Compose

- **Component 4 – Docker hub Registry**

Action	Purpose
Push Image	Store container image

Action	Purpose
Pull Image	Used during deployment

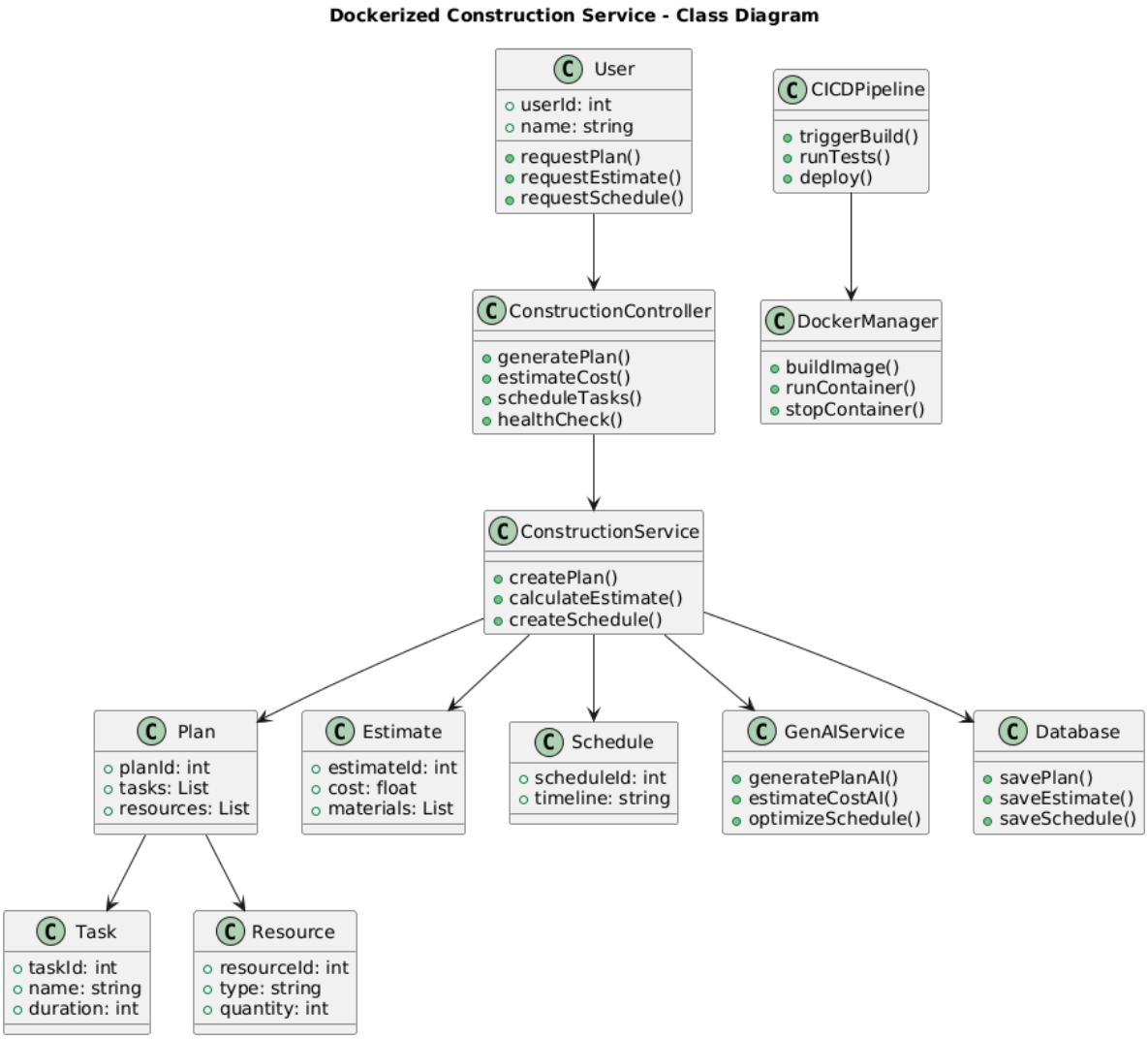

Figure 3: Class Diagram

2.5. Key Design Considerations

1. Multi-stage Docker build

- Reduces image size
- Improves security
- Removes unnecessary files

2. Separation of Concerns

- Application code and DevOps configs separated
- Enables parallel development

3.Environment Consistency

- Same container runs in dev, test, prod
- Eliminates "works on my machine" problem

4.GenAI Integration

- AI assists in construction planning
- Keeps logic modular and scalable

5.Security

- Minimal base images
- No secrets stored in image
- Environment variables used

6. Cost Efficiency

- Lightweight containers
- Minimal infrastructure usage

2.6. API Catalogue

1. Internal APIs

Endpoint	Method	Purpose
/health	GET	Health check
/plan	POST	Generate construction plan
/estimate	POST	Cost estimation

Endpoint	Method	Purpose
/schedule	POST	Task scheduling

2. External APIs

API	Purpose
GenAI API (LLM)	Intelligent planning & suggestions

Authentication:

- API key via environment variables

3. References

- Docker Documentation
- GitHub Actions / Jenkins Docs
- GenAI API Documentation

4. GitHub

- Construction Service Repo
- DevOps Infrastructure Repo